Understanding the Alab Dual-Camera Computer Vision System

A practical guide to what the cameras see, how space is represented, how the purple tape and blue joints are used, and what to run today.

Repository: github.com/JessN777/Computer-vision-Alab
Workspace layout: robotics/ (Python CV tools) + ros2_ws/ (ROS 2 dual_view stack)
Hardware: two Orbbec Gemini-class RGB-D cameras + UR arm cell
Audience: you - operator / developer trying to see the big picture

What this guide covers

- 1. Big picture: two eyes on one cell
- 2. Hardware and software map
- 3. Coordinate frames (ur_base, cameras, purple tape)
- 4. Why tape is not ArUco - and what we do instead
- 5. Scene + purple-tape extrinsics (new path)
- 6. Live dual_view: furnace, rack, blue joints, Kalman
- 7. Vial / depthcamera stack (separate but related)
- 8. Files that matter and what "ready" means
- 9. How to run things day-to-day
- 10. Troubleshooting and what "good" looks like

1. Big picture: two eyes on one cell

The lab cell has a Universal Robots (UR) arm, a rack, a furnace area, and other equipment. Two Orbbec RGB-D cameras are mounted at roughly right angles so they look into the same workspace from different sides. Each camera gives a color image and a depth map (millimeters to each pixel on opaque surfaces).

Computer vision here is not one monolith. It is several cooperating jobs:

- Watch the arm: find blue joint blobs, keep IDs when they leave the frame.
- Know where "home" is: purple tape / pedestal as a shared origin.
- Know where each camera sits in the cell (extrinsics / cell_frames).
- Overlay furnace and rack state (OPEN/CLOSED style ROIs).
- Optionally track clear vials with white caps (depthcamera / YOLO path).

Mental model

Think of cam0 and cam1 as two people standing at the corners of a room. They see the same furniture from different angles. If you know how those two people are oriented relative to each other (and to the robot base), you can say where an object is in one shared room coordinate system - called ur_base in this project.

What "spatially understanding" means

For a point seen by a camera at pixel (u,v) with depth Z millimeters, the camera can convert that to a 3D point in its own frame (X right, Y down, Z forward - OpenCV convention). To compare cam0 and cam1, or to talk about the robot base, you need a rigid transform: rotation R and translation t.

```
p_base = R @ p_cam + t_mm     # cell_frames.py
```

If R and t are wrong, the two cameras disagree about where the same physical joint is. That is why purple tape, scene matching, and (optionally) a calibration board all exist: they are ways to get R and t right.

2. Hardware and software map

Hardware

- Two Orbbec RGB-D cameras (USB 3 - each on its own port when possible).
- UR arm with blue-highlighted joints (HSV "baby blue" blobs).
- Purple tape near the pedestal / base - shared visual origin.
- Rack and furnace regions used by overlay classifiers.

Software trees

Computer-vision-Alab splits into two main folders after the Jessica split plan:

```
Computer-vision-Alab/
  robotics/          # plain Python: calib, vial tools, cell_frames.json
  ros2_ws/           # ROS 2 + Pixi: orbbec_dual_sync dual_view live stack
```

Key live ROS package: orbbec_dual_sync

- camera_publisher - publish RGB/depth from each Orbbec.
- dual_view_node - side-by-side preview: furnace | rack + arm overlays.
- baby_blue_draw - blue joint / purple tape HSV helpers.
- blue_joint_tracks - 6-slot joint bank + Kalman coast in ur_base.
- cell_frames - load/save camera poses in the cell.
- joint_motion_net - small motion net to keep predicted boxes on screen.

Key robotics (non-ROS) tools

- scene_extrinsics_calib.py - NEW: ArUco-free R,t from scene + tape.
- stereo_calib_stub.py - classic ChArUco/chessboard path (optional).
- depthcamera.py - vial distance / dual-independent locate.
- cell_frames.json / stereo_extrinsics.json - saved geometry.

Environment tip

ROS launches expect ORBBEC_CAMERA_ROOT to point at the robotics/ folder so nodes can import depthcamera / read JSON configs. If imports fail, check that variable first.

3. Coordinate frames (what is "where")

ur_base - the shared world

The world origin (0,0,0) is defined at the robot base / blue pedestal idea. In the live stack, purple tape is treated as the shared XY origin for joint polar coordinates (distance and angle from the tape). cell_frames.json stores each camera's R and t_mm so that camera points can be mapped into ur_base.

Camera optical frames

Each Orbbec has its own frame: +X to the right in the image, +Y down, +Z out of the lens. Depth values are along +Z for that camera. Clear plastic (vial cups) often returns bad depth; opaque lids and blue joints are more trustworthy.

Current default layout: perpendicular_90

Today's cell_frames.json uses layout=perpendicular_90: cam0 has identity rotation; cam1 is assumed yawed +90 degrees about Y (cameras looking inward at right angles). Translations t_mm were bootstrapped so the pedestal maps near the origin. That is a strong prior - often good enough to start - but it is not a full measured calibration.

```
layout: perpendicular_90
cam0: R = I,  t bootstrapped from pedestal
cam1: R = yaw+90, t bootstrapped from pedestal
```

After scene+tape calib: layout=scene_tape

When you run scene_extrinsics_calib.py successfully, the layout name becomes scene_tape. The assumed 90 degree R is replaced by a solve from real matched 3D points. Purple tape anchors the origin in both views.

Stereo file vs cell_frames

- stereo_extrinsics.json - maps points from cam0 into cam1: X1 = R X0 + t.
- cell_frames.json - maps each camera into ur_base (robot-centric).

They describe the same geometry in different packaging. Scene calib writes both so vial reproject assist and dual_view joint fusion can stay consistent.

4. Purple tape, ArUco, and what failed

What you tried with tape

Colored tape is easy to put in the cell. The program can find a purple blob with HSV thresholds. That works well for "where is the origin spot?" It works poorly if you ask the program to read the full 3D orientation of a short shiny strip the way ArUco corners would.

What ArUco / ChArUco would give you

A printed board with unique corner IDs gives many known 3D-2D correspondences in both cameras. OpenCV stereoCalibrate then returns accurate R and t. That path still exists (stereo_calib_stub.py) but is hard when the arm fills the overlap and you cannot place a board.

What purple tape IS good for

- Centroid: a named point both cameras can agree is "home" (0,0).
- Long axis: the strip is a LINE - that helps lock yaw (heading), not full 6-DoF alone.
- Joint polars: r_mm and ang_deg of blue joints relative to the tape origin.

What purple tape is NOT

- Not a substitute for many scene point matches when solving full camera R,t.
- Not reliable if you treat strip "which way is up" as a precise pose by itself.
- Not required for every feature - pedestal blue base can also bootstrap t.

Rule of thumb

Use tape as a bright landmark (point + line). Use the rest of the static scene (rack edges, fixtures) as many point pairs. Together they replace ArUco without needing a printed board in the arm's way.

5. Getting R,t - board vs scene (including asymmetric checkerboard)

You have three supported ways to learn where cam1 sits relative to cam0. All write the same files: stereo_extrinsics.json and cell_frames.json.

Path A - Asymmetric checkerboard (what you are trying now)

Yes - an asymmetric checkerboard is a first-class option. "Asymmetric" here means the board is NOT square in corner count (for example 9x6 inner corners, not 8x8). That breaks the 180-degree ambiguity of a square board, so OpenCV can lock orientation reliably.

- Hold the board so BOTH Orbbecs see it (park the arm if needed).
- Measure one square in millimeters with a ruler.
- Capture many dual pairs, then calibrate.

```
python stereo_calib_stub.py capture
python scene_extrinsics_calib.py board --asymmetric-chess --cols 9 --rows 6 --square-mm 25
python scene_extrinsics_calib.py status
```

There is also an asymmetric CIRCLE grid (staggered dots) if that is the pattern you printed: --asymmetric-circles --cols 4 --rows 11 --square-mm 20.

Path B - Scene + purple-tape (no board)

When the arm blocks a board, match the shared scene with ORB + depth and use purple tape as origin + long-axis yaw lock.

```
python scene_extrinsics_calib.py solve --live
```

Path C - ChArUco (optional classic)

Printed ChArUco still works via stereo_calib_stub.py calibrate --charuco.

Which should you use today?
If you can place an asymmetric checkerboard in the dual overlap for a few minutes: use Path A (board). If the arm permanently fills that space: use Path B (scene+tape). Purple tape still helps Path B for origin/yaw; the board replaces tape for the heavy R,t solve when available.

Scene + tape algorithm (Path B detail)

This ArUco-free path answers: given what both cameras already see from different angles, where is cam1 relative to cam0?

Algorithm (plain language)

- 1. Grab synced RGB + depth from both cameras.
- 2. Mask out blue arm pixels so moving links are not used as landmarks.
- 3. Detect ORB features on static structure; match descriptors across views.
- 4. For each match, read depth and unproject to 3D millimeters.
- 5. Find purple tape in both views: centroid + long-axis direction.
- 6. Weight tape center (and axis endpoints) heavily in the point cloud.

- 7. RANSAC + Umeyama: find rigid R,t mapping cam0 points into cam1.
- 8. Refine yaw so transformed tape axis aligns with the other camera's axis.
- 9. Write stereo_extrinsics.json and update cell_frames (layout=scene_tape).

Commands

```
cd robotics
python scene_extrinsics_calib.py howto
python scene_extrinsics_calib.py solve --live
  # SPACE = buffer a dual frame, S = solve + write, Q = quit
python scene_extrinsics_calib.py status
python -m pytest test_scene_extrinsics.py -v
```

When the solve fails

- Not enough overlap / no texture - point cameras more into shared structure.
- Arm filling the view - park it; masking helps but empty static FOV is better.
- Tape missing in one camera - fix lighting / HSV; longer matte purple strip helps.
- Depth holes - prefer opaque surfaces; avoid trusting glass.

Success looks like: status shows non-PLACEHOLDER stereo notes, layout=scene_tape, and the same physical landmark (tape center) maps near the same ur_base XYZ from both cameras.

6. Live dual_view: what you see on screen

dual_view_node builds a side-by-side preview. It subscribes to camera topics (it does not open CAP_OBSENSOR itself). Left pane is typically furnace-oriented CV; right pane rack OPEN/CLOSED style overlays. Arm / joint drawing is layered using baby_blue helpers and the joint bank.

Blue joints + Kalman coast

Blue joint detections are HSV blobs with depth lift into camera frame, then into ur_base via cell_frames. A fixed-size bank (default 6 tracks) associates detections over time. When a joint leaves the camera, Kalman prediction keeps the highlight and angle label for several seconds ("coast"). A small motion network can also help predicted boxes stay plausible.

Purple tape in the live stack

find_purple_tape returns a box (+ optional depth). Joint tracks report polar coordinates relative to that origin: distance in the XY plane and yaw angle. If the tape flickers, a short hold keeps the last origin. Scene calib extends this idea with an explicit long-axis measurement for orientation locking.

Furnace and rack overlays

These are mostly ROI / color / geometry overlays configured by JSON (furnace and rack configs under robotics or package share). They answer operational questions (e.g. rack closed) and are separate from the extrinsics solve - but they share the same camera images.

Independence note

Joint tracking can look "alive" even when stereo_extrinsics.json is still a PLACEHOLDER, because perpendicular_90 + pedestal bootstrap already provide a usable ur_base. Scene calib improves agreement between cameras; it is not required for every overlay to draw.

7. Vial / depthcamera stack (related sibling)

Separately from dual_view arm overlays, robotics/depthcamera.py tracks clear vials with white screw caps. Clear cups break depth; distance is taken from the opaque white lid. A YOLO model proposes boxes; classical gates (clear-vial hole under the lid, scale at Z, sideways assist) accept or reject.

Dual vial modes

- --dual-independent (preferred): two processes; localhost IPC only corroborates cm.
- --dual / locate_pair: older sticky peer soft-assist path.
- --3d-reproject-assist: if stereo_extrinsics is REAL, propose a box on the misser.

Scene+tape calib is exactly what makes --3d-reproject-assist trustworthy. While stereo_extrinsics.json says PLACEHOLDER, reproject assist refuses to invent boxes.

How this connects to the arm cell

Same two cameras, different jobs. Arm dual_view cares about joints and cell frames. Vial tools care about white caps and per-camera centimeters. Shared geometry files (stereo_extrinsics / cell_frames) are the bridge when you want cross-camera geometric assist.

8. Files that matter right now

Geometry state

- robotics/cell_frames.json - ur_base poses; layout perpendicular_90 or scene_tape.
- robotics/stereo_extrinsics.json - cam0->cam1; PLACEHOLDER until calibrated.

Calib tools

- robotics/scene_extrinsics.py - library: ORB, tape axis, Umeyama/RANSAC.
- robotics/scene_extrinsics_calib.py - CLI live/file solve.
- robotics/stereo_calib_stub.py - optional board path + howto pointer.
- robotics/test_scene_extrinsics.py - unit tests (no cameras needed).

Live dual_view

- ros2_ws/.../dual_view_node.py - main preview node.
- ros2_ws/.../blue_joint_tracks.py - joint bank + tape origin polars.
- ros2_ws/.../baby_blue_draw.py - HSV blue + purple tape (+ long-axis helper).
- ros2_ws/.../cell_frames.py - ROS import of the frame graph.
- robotics/joint_motion_mlp.npz - weights for motion coast assist.

Vial (optional)

- robotics/depthcamera.py - live vial distance.
- robotics/vial_3d_model.py - dims + extrinsics loaders.

GitHub status (as of this guide)

Canonical repo: JessN777/Computer-vision-Alab. Scene+tape work was pushed via fork Andrea-gm and opened as Pull Request #1 because collaborator accounts had pull-only access to main. Merge that PR (or grant write access) to land scene_extrinsics_* on main.

9. Day-to-day runbook

A. Improve camera spatial agreement (recommended)

```
1. Park or slow the UR arm.
2. Ensure purple tape is visible to BOTH Orbbecs.
3. cd robotics
4. python scene_extrinsics_calib.py solve --live
5. SPACE a few varied dual frames, then S.
6. python scene_extrinsics_calib.py status
```

B. Live dual_view overlays

Use your usual launch scripts under ros2_ws (Pixi + launch_*.cmd patterns). Set ORBBEC_CAMERA_ROOT to the robotics folder. Start camera publishers, then dual_view. You should see furnace/rack panes and blue joint highlights with angles relative to the purple origin when tape is found.

C. Vial distance (optional)

```
cd robotics
python depthcamera.py --dual-independent
python depthcamera.py --dual-independent --3d-reproject-assist
```

10. Troubleshooting cheat sheet

Joints jump between cameras / disagree in 3D

Re-run scene+tape calib; verify cell_frames ready flags; check tape visibility.

Purple tape not found

Lighting and HSV band (magenta). Try a longer matte purple strip. Avoid strong red clutter near the pedestal. Use _hsv_probe helpers under arm_state if needed.

Reproject assist does nothing

stereo_extrinsics still PLACEHOLDER - run scene_extrinsics_calib or board calib until status is clean.

Imports / ROS cannot find robotics

Export ORBBEC_CAMERA_ROOT to the absolute path of robotics/.

What "good" looks like

- Tape visible; joint labels stable; coast works when a joint leaves view.
- status: non-placeholder stereo; cell_frames layout=scene_tape after calib.
- Same landmark within a few centimeters from both cams in ur_base.

One-sentence summary

Two depth cameras watch the UR cell; purple tape marks home; blue blobs are joints; cell_frames puts everyone in ur_base; scene+tape matching (not ArUco) is how you teach the computers where they sit relative to each other when the arm is in the way.

End of guide. Rebuild this PDF anytime with:

```
python robotics/docs/build_cv_guide_pdf.py
```